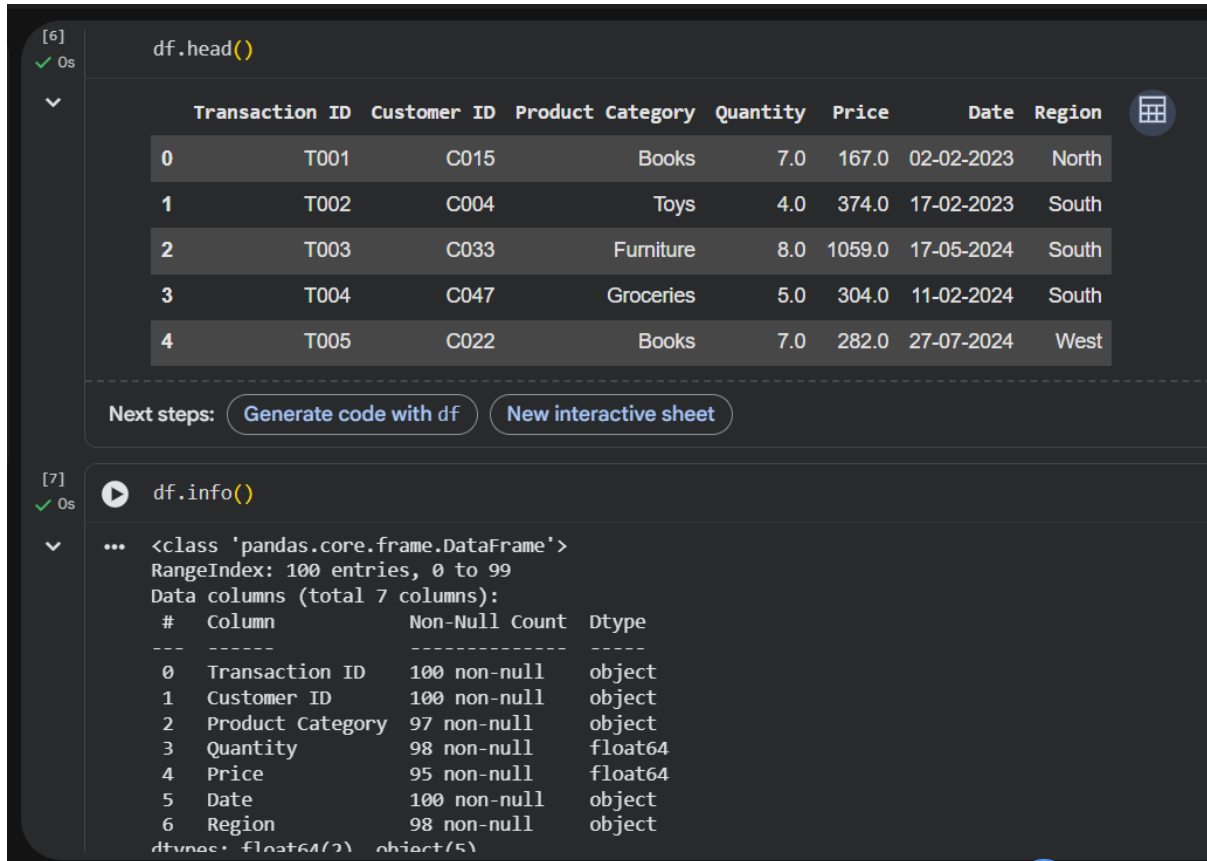


Libraries: NumPy & Pandas Assignment

Q1. Load the given retail sales dataset into a Pandas DataFrame and display the first 5 rows along with the dataset information to understand its structure.

Answer:

We have Load the CSV into a Data Frame { `df = pd.read_csv('retail_dataset.csv')` }



The screenshot shows a Jupyter Notebook interface with two code cells. The first cell, labeled [6], contains the code `df.head()` and displays the first 5 rows of the dataset. The second cell, labeled [7], contains the code `df.info()` and displays the dataset's structure, including the number of entries, column names, and data types.

df.head()

	Transaction ID	Customer ID	Product Category	Quantity	Price	Date	Region
0	T001	C015	Books	7.0	167.0	02-02-2023	North
1	T002	C004	Toys	4.0	374.0	17-02-2023	South
2	T003	C033	Furniture	8.0	1059.0	17-05-2024	South
3	T004	C047	Groceries	5.0	304.0	11-02-2024	South
4	T005	C022	Books	7.0	282.0	27-07-2024	West

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   Transaction ID         100 non-null   object
1   Customer ID           100 non-null   object
2   Product Category      97 non-null    object
3   Quantity              98 non-null    float64
4   Price                 95 non-null    float64
5   Date                  100 non-null   object
6   Region                98 non-null    object
dtypes: float64(2), object(5)
```

Q2. Inspect the dataset and identify missing values in each column. Display the total number of missing values present in the dataset.

Answer:



The screenshot shows a Jupyter Notebook interface with a code cell labeled [8] containing the code `df.isnull().sum()`. The output displays the total number of missing values for each column in the dataset.

df.isnull().sum()

Column	Count
Transaction ID	0
Customer ID	0
Product Category	3
Quantity	2
Price	5
Date	0
Region	2

df.isnull()

	Transaction ID	Customer ID	Product Category	Quantity	Price	Date	Region
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...
95	False	False	False	False	False	False	False
96	False	False	False	False	False	False	False
97	False	False	False	False	False	False	False
98	False	False	False	False	False	False	False
99	False	False	False	False	False	False	False

100 rows x 7 columns

Q3. Replace missing values in the Quantity and Price columns with their respective mean values using Pandas.

Answer:

For Quantity Column

```
df['Quantity'] = df['Quantity'].fillna(df['Quantity'].mean())
```

df['Quantity']

	Quantity
0	7.0
1	4.0
2	8.0
3	5.0
4	7.0
...	...
95	10.0
96	9.0
97	7.0
98	9.0
99	8.0

100 rows x 1 columns

For Price Column

```
df['Price'] = df['Price'].fillna(df['Price'].mean())
```

df['Price']

	Price
0	167.0
1	374.0
2	1059.0
3	304.0
4	282.0
...	...
95	2901.0
96	1077.0
97	643.0
98	429.0
99	1183.0

100 rows x 1 columns

dtype: float64

Q4. Remove all rows where the Product Category or Region column contains missing values.

Answer:

```
df = df.dropna (subset=['Product Category', 'Region'])

df.shape

(95, 7)
```

Q5. Create a new column called Revenue using NumPy, where: $\text{Revenue} = \text{Quantity} \times \text{Price}$. Display the updated dataset with the new column.

Answer:

```
[19] import numpy as np

[20] df['Revenue'] = np.multiply(df['Quantity'], df['Price'])

... /tmp/ipykernel_3913/1830166690.py:11: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df['Revenue'] = np.multiply(df['Quantity'], df['Price'])

[23] df.head()
```

	Transaction ID	Customer ID	Product Category	Quantity	Price	Date	Region	Revenue
0	T001	C015	Books	7.0	167.0	02-02-2023	North	1169.0
1	T002	C004	Toys	4.0	374.0	17-02-2023	South	1496.0
2	T003	C033	Furniture	8.0	1059.0	17-05-2024	South	8472.0
3	T004	C047	Groceries	5.0	304.0	11-02-2024	South	1520.0
4	T005	C022	Books	7.0	282.0	27-07-2024	West	1974.0

Q6. Using NumPy, calculate the total revenue generated from all transactions in the dataset.

Answer:

```
Total_Revenue = np.sum(df['Revenue'])
print(Total_Revenue)

465529.66058002145
```

Q7. Group the dataset by Product Category and calculate the total revenue generated by each category.

Answer:

```
category_revenue = df.groupby('Product Category')['Revenue'].sum().sort_values(ascending=False)
print(category_revenue)
```

```
Product Category
Furniture      129664.526316
Electronics    93204.315789
Clothing       85006.000000
Toys           53947.879699
Sports         34484.000000
Books          29597.938776
Beauty         27647.000000
Groceries      11978.000000
Name: Revenue, dtype: float64
```

Q8. Based on the revenue calculated for each product category, identify the top 3 categories with the highest revenue and the bottom 3 categories with the lowest revenue.

Answer:

Top 3 Categories

```
category_revenue = df.groupby('Product Category')['Revenue'].sum().sort_values(ascending=False)

top_3 = category_revenue.head(3)
print(top_3)
```

Product Category	Revenue
Furniture	129664.526316
Electronics	93204.315789
Clothing	85006.000000

Name: Revenue, dtype: float64

Bottom 3 Categories as well

```
bottom_3 = category_revenue.tail(3)
print(bottom_3)
```

Product Category	Revenue
Books	29597.938776
Beauty	27647.000000
Groceries	11978.000000

Name: Revenue, dtype: float64

Q9. Group the dataset by Region and calculate the total revenue generated in each region. Identify the region with the highest revenue and the region with the lowest revenue.

Answer:

Grouped by Regions

```
region_revenue = df.groupby('Region')['Revenue'].sum().sort_values(ascending=False)
print(region_revenue)
```

Region	Revenue
North	153244.353383
East	126232.044039
South	104855.736842
West	81197.526316

Name: Revenue, dtype: float64

Highest Revenue

```
highest_region = region_revenue.idxmax()
highest_value = region_revenue.max()
print("Highest Revenue Region:", highest_region, "-", highest_value)
```

Highest Revenue Region: North - 153244.35338345866

Lowest Revenue

```
lowest_region = region_revenue.idxmin()
lowest_value = region_revenue.min()
print("Lowest Revenue Region:", lowest_region, "-", lowest_value)
```

Lowest Revenue Region: West - 81197.52631578947

Q10. Extract the month from the Date column using Pandas and group the dataset by month to calculate the total monthly revenue. Also, calculate the mean, median, and standard deviation of the Revenue column using NumPy.

Answer:

```
[35]
✓ 0s
monthly_revenue = df.groupby('Month')['Revenue'].sum()
print(monthly_revenue)

Month
1      43994.000000
2      33028.000000
3      10557.000000
4      57298.526316
5      27915.000000
6      28190.000000
7      47851.292159
8      76958.000000
9      30719.105263
10     38017.000000
11     27662.736842
12     43339.000000
Name: Revenue, dtype: float64

[36]
✓ 0s
mean_revenue = np.mean(df['Revenue'])
median_revenue = np.median(df['Revenue'])
std_revenue = np.std(df['Revenue'])

print("Mean Revenue:", mean_revenue)
print("Median Revenue:", median_revenue)
print("Standard Deviation of Revenue:", std_revenue)

Mean Revenue: 4900.312216631804
Median Revenue: 3282.1052631578946
Standard Deviation of Revenue: 4721.171795257337
```